Hierarchical DBMS

Data models in database systems

Lecture 7

Mara Romanovska
Mara.Romanovska@rtu.lv

20.11.2025.

1862
RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

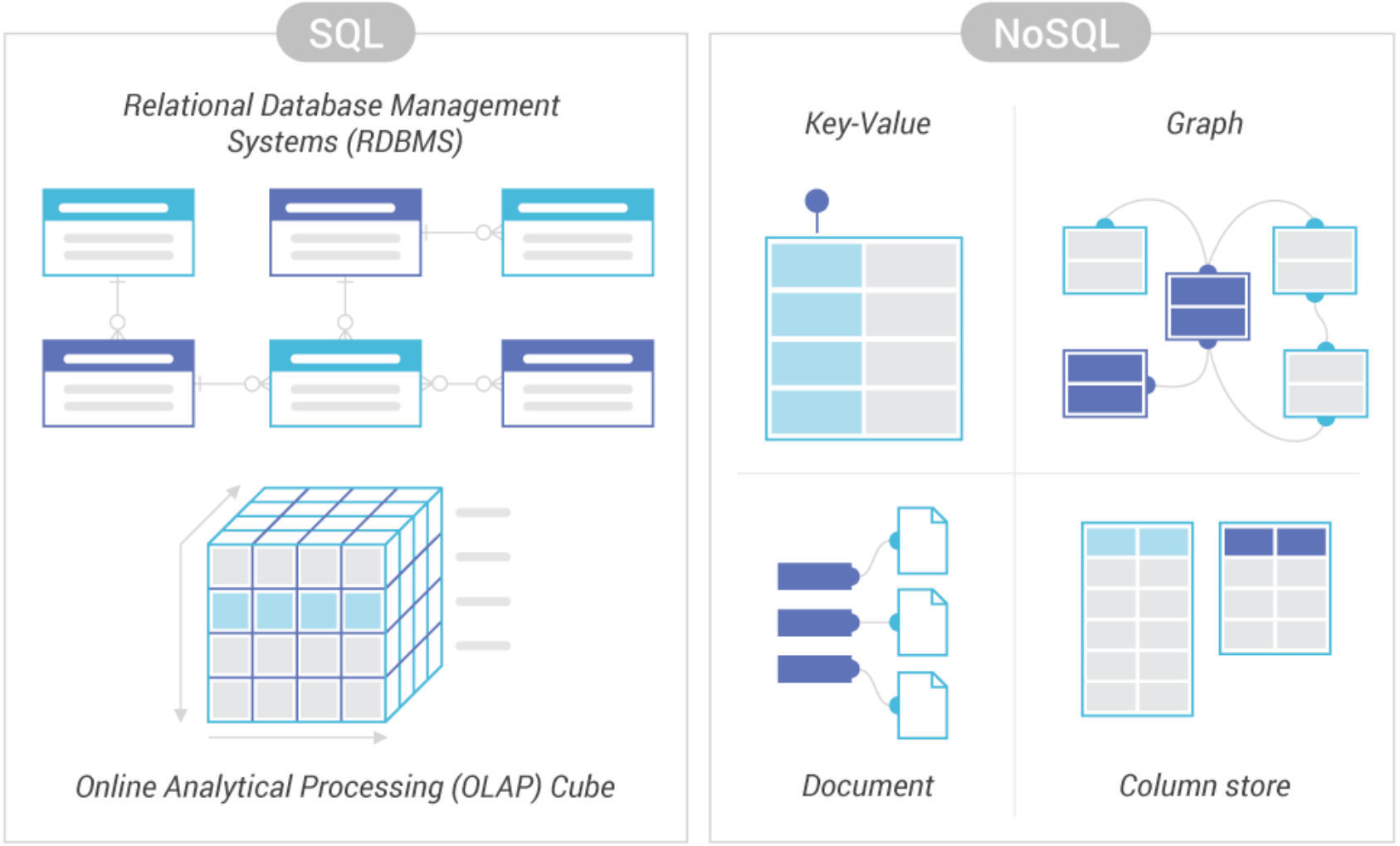
Content

- Storing semi-structured data
- Hierarchical data model
- XML data format
- XML schema
- Querying XML

Feedback on midterm test

- Many of you did not read tasks
  - «Create a **collection**…»
  - «Create a **table/type**…»
  - «Name **dimensions!**»
- Exam is going to be more extensive and summative
  - Pay attention to points you get for the task; they correlate with complexity
- Some notes on concepts
  - Use of warehousing and normalization
  - Database optimization – column store vs. table with object column
- Consultation today after class, 16:15, Z10-404

Options for data logical organisation



https://www.scylladb.com/resources/nosql-vs-sql/

Problems with relational data models

- The relational model is a good approach for data that are **highly structured in their nature**

Name	DeathYear	Genre	Country	Primary Instrument	Website
David Bowie	2016	Rock / Art Rock	United Kingdom	Vocals	NULL
Beyoncé Knowles	NULL	Pop / R&B	United States	Vocals	https://www.beyonce.com
Daft Punk	NULL	Electronic	France	NULL	NULL
Freddie Mercury	1991	Rock	United Kingdom	Vocals, Piano	NULL
Taylor Swift	NULL	Pop / Country	United States	Guitar	https://www.taylorswift.com
Miles Davis	1991	Jazz	United States	Trumpet	NULL
Björk	NULL	Art Pop / Avant	Iceland	Vocals	NULL

Semi-structured data idea

- There are plenty use-cases where data is
  - not necessarily well structured
  - imposing such a structure is inefficient

- There is, however, **some structure**

An example: CVs

EMILY WILLIAMS

Bridgeport, CT 06606 ▪ (555) 555-5555 ▪ example@example.com

PERSONAL SUMMARY

Focused and attentive, recently graduated Ph.D. student, Assistant Professor of English experienced in cultivating welcoming and engaging learning environments. Friendly and personable with the skill to stay organized and on top of important deadlines. Dedicated to the development of an English curriculum reflective of the needs of students and work environments. Strong knowledge of teaching methods and techniques.

CORE QUALIFICATIONS

- Writing coursework
- Student needs assessment
- Class instruction
- Student records management
- Student research guidance
- MS Office expertise
- Outlook and Gmail
- Tutoring

EDUCATION

Ph.D. : English Language And Literature
Yale University - New Haven, CT

Master of Arts : Writing
Albertus Magnus College - New Haven, CT

Bachelor of Arts : English
Southern Connecticut State University - New Haven, CT

WORK EXPERIENCE

ASSISTANT PROFESSOR OF ENGLISH 09/2019 to Current
Housatoni Community College, Bridgeport, CT

- Provided guidance and supervision to 20 master's degree students while giving academic support to Professors and other faculty members.
- Took attendance, graded assignments, and maintained student records to assist Professors with administrative tasks and maintain smooth daily operations.
- Created materials and exercises to illustrate the application of course concepts.

ASSISTANT ENGLISH INSTRUCTOR 09/2016 to 06/2019
Sacred Heart University, Fairfield, CT

- Evaluated college students' abilities and grasp of English language, keeping appropriate records and preparing progress reports.
- Evaluated and revised lesson plans and course content to facilitate and moderate classroom discussions and student-centered learning.
- Lectured and enabled for four bi-weekly sections of Literature 101.

WRITING FELLOW 09/2015 to 06/2016
Fairfield University, Fairfield, CT

MARC LARSSON

PROFESSIONAL GRAPHIC DESIGNER

820 Colorado, California, CA - 85210 | +00 (123) 4567 - 890 | markmartin@email.com | www.markmartines.com

ABOUT ME

Lorema ipsumzie sit dolor consectetuer adipiscing elit secure is euismod tincidunt but laoreet dolore magna aliquam erat volutpat. Enim ads minimalism quickly of only exercitation ullacorp suscipit lobortis nislanicts but excepts commodo consequat. Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit lobortis nisla but aliquip except commodo.

EDUCATION

Website Developments 1993 - 1996
NEVADA International University

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, but diam nonummy nibh euismod tincidunt.

Graphics & Art Designer 1996 - 1999
DOCOMO International University

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, but diam nonummy nibh euismod tincidunt.

WORK EXPERIENCES

2000 - 2008 / Graphic and Print Designer
SLOANE International Advertising

Lorem ipsum dolor sit amet, is consectetuer adipiscing elit, but anyone of nonummy nibh euismod tincidunt.

2008 - 2016 / Website Designer
DOCOMO Real Estate Agency

Lorem ipsum is dolor sit amet, consectetuer adipiscing elite, but anyone diam nonummy nibh euismod tincidunt.

PRO - SKILL

- 3D ANIMATIONS
- GRAPHIC DESIGN
- WEB DESIGN
- APP DEVELOPMENT
- PHP / JQUERY
- WORDPRESS
- COMMUNICATION
- TEAMWORK
- MANNAGEMENT

REFERENCES

George Emanuells
Founder of Stand Real Estate
+00 (123) 4567 890
founder@standestate.com

CV - RESUME

STUDENT CV

BY RESUME GENIUS

Detail-oriented student journalist with a track record of publishing thought-provoking articles in the university newspaper. Proficient in fact-checking, editing, and meeting tight deadlines. Aiming to hone expertise in long-form narrative journalism and documentary filmmaking.

(973) 538-5492 | linkedin.com/in/your-profile
your.email@email.com | New York, NY

EDUCATION

NEW YORK UNIVERSITY, NEW YORK, NY
Bachelor of Arts in Journalism
Expected graduation: May 20XX
GPA: 3.8/4.0
Dean's list for 4 consecutive semesters

Relevant Coursework: Media Ethics and Law, News Writing and Reporting, Feature Writing, Investigative Journalism, Digital Storytelling, Multimedia Production, Copy Editing and Fact-Checking, Narrative Journalism Workshop, Data Journalism

WORK EXPERIENCE

THE DAILY TRUBINE, New York, NY
Editorial Intern
May 20XX–present

- Assist in researching and fact-checking for feature articles and investigative pieces
- Wrote and published 5 news articles and 2 human interest stories for both print and online editions
- Conduct interviews with local community leaders and experts
- Collaborate with senior editors to revise and polish long-form narrative pieces

UNIVERSITY HERALD, New York, NY
Staff Writer
January 20XX–Present

- Write weekly articles on campus events, student life, and academic achievements
- Pitch and develop story ideas independently, meeting tight deadlines consistently
- Edit and proofread articles for fellow staff writers
- Participate in weekly editorial meetings to plan content and discuss journalistic ethics

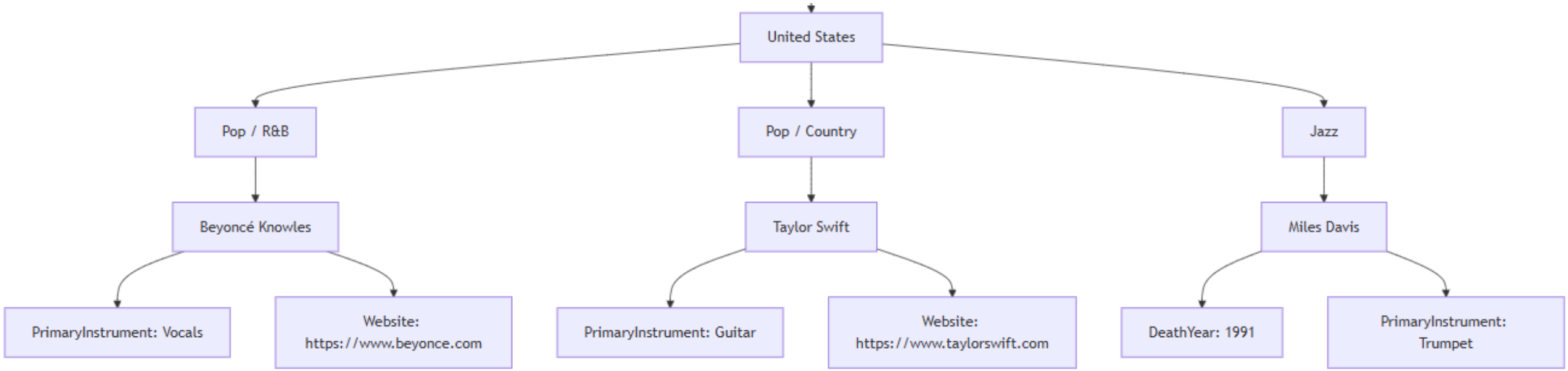
Schema and semi-structured data

- What is schema in relational databases?
- Sometimes semi-structured data are called **schemaless** – but that is not entirely true
- **Schema-on-read** not **schema-on-write**

- Shredding in the context of semi-structured data means creating a strict schema for **hierarchical data model**

Semi-structured data implementation

- Done with hierarchical data model
- Example from the previous table (a fragment):



Semi structured data implementation

- Data is encoded in some **standart format**, such as:
  - XML
  - YAML
  - JSON
  - BSON

Criteria	XML	JSON
Type	Markup language – document-oriented	Data exchange format – data oriented
Structure definition	Non-predefined meaning for the structure	Semi-predefined meaning for the structure
Structured data	Deals poorly	Deals good with structured data
Schema definition	Extensive schemas and constraints can be set	Some constraints can be set

Semi structured data implementation

XML

```xml
<MusicWorld>
    <Country name="United States">
        <Genre name="Pop / R&B">
            <Musician name="Beyoncé Knowles">
                <PrimaryInstrument>Vocals</PrimaryInstrument>
                <Website>https://www.beyonce.com</Website>
            </Musician>
        </Genre>

        <Genre name="Pop / Country">
            <Musician name="Taylor Swift">
                <PrimaryInstrument>Guitar</PrimaryInstrument>
                <Website>https://www.taylorswift.com</Website>
            </Musician>
        </Genre>

        <Genre name="Jazz">
            <Musician name="Miles Davis">
                <DeathYear>1991</DeathYear>
                <PrimaryInstrument>Trumpet</PrimaryInstrument>
            </Musician>
        </Genre>
    </Country>
</MusicWorld>
```

JSON

```json
{
  "MusicWorld": {
    "Country": {
      "name": "United States",
      "Genres": [
        {
          "name": "Pop / R&B",
          "Musicians": [
            {
              "name": "Beyoncé Knowles",
              "PrimaryInstrument": "Vocals",
              "Website": "https://www.beyonce.com"
            }
          ]
        },
        {
          "name": "Pop / Country",
          "Musicians": [
            {
              "name": "Taylor Swift",
              "PrimaryInstrument": "Guitar",
              "Website": "https://www.taylorswift.com"
            }
          ]
        },
        {
          "name": "Jazz",
          "Musicians": [
            {
              "name": "Miles Davis",
              "DeathYear": 1991,
              "PrimaryInstrument": "Trumpet"
            }
          ]
        }
      ]
    }
  }
}
```

Hierarchical data model and document data bases

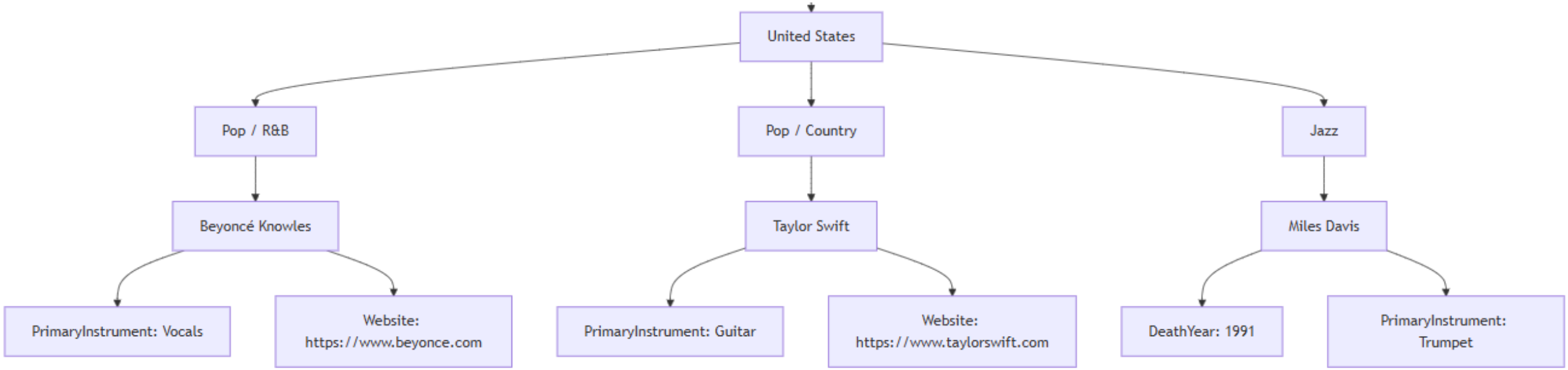
- Hierarchical data model is the basis of document databases (DDBs)
- Designed for storing, retrieving and managing **document-oriented information**, also known as semi-structured data
- One of the main categories of NoSQL databases

Typical applications

- Used when:
  - Data changes frequently
  - Records do not share identical fields
  - Rigid schemas would slow down development
  - Interoperability across systems is important
- Some applications:
  - Social media, especially when integrating various systems
  - Healthcare (e.g., lab results)
  - CVs and other typical documents

Hierarchical data model

Hierarchical data model characteristics



- Data **redundancy** is present
  - Where do trumpet properties go?
- Information on object is stored **in one place**

Aspects of HDM (as compared to RDM)

Drawbacks

- Worse support for joins
- Worse support for many-to-many relationships

- Cannot go directly to nested element, must know position
  - Not a problem if documents are not deeply nested
- In general – depends on the data model
  - if the systems are extremely complex, coding is cumbersome

Benefits

- Schema flexibility
- Closer to data structure of the domain
- Sometimes better shows innate structure of the data

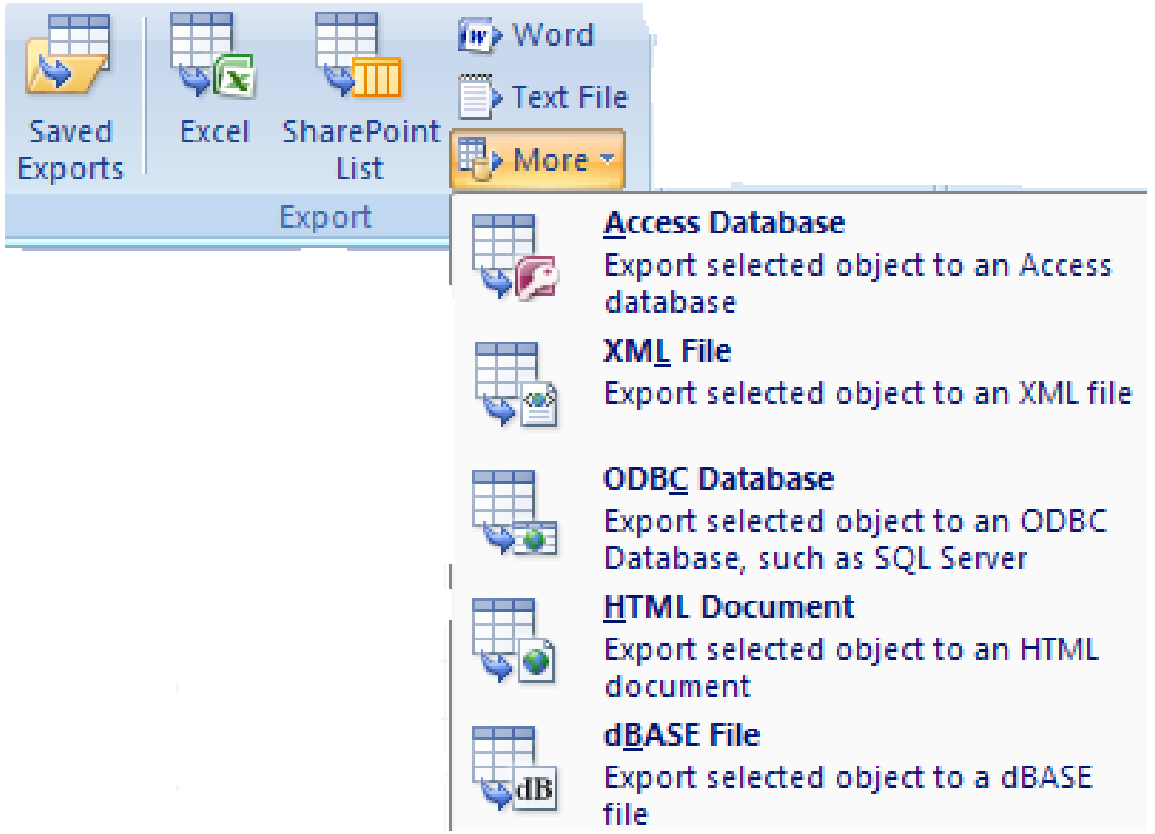
XML data format

https://www.db-book.com/online-chapters-dir/30.pdf

XML (Extensible Markup Language)

- Extensible Markup Language (XML) was not designed for database applications
- Has it’s roots in document management
- Designed to represent data
- Useful as common data format

Example of XML as export format



Markup idea in general

- Marks anything not intended for output
- A markup language is a formal description of
  - what part of the document is content
  - what part is markup
  - what the markup means

- Simplest example: HTML

Benefits of markup languages

- For documents:
  - allows the document to be formatted differently in different situations
  - allows to be formatted in a uniform manner

- Record what each part of the text represents semantically
- Helps to automate extraction of key parts of documents

Tags

- The **essential implementation basis** of the markup
- Tags provide context for each value and allow the semantics of the value to be identified

- Tags are used in pairs with <tag> and </tag>

<title> *Data base data models* </title>

- **XML does not prescribe the set of tags allowed**

XML example

```
<?xml version="1.0" encoding="UTF-8" ?>
- <customer_order number="004985" date="2004-06-24">
  - <lines>
    - <line no="1">
        <item>Disc CD</item>
        <quantity>30</quantity>
        <price>0.95</price>
      </line>
    - <line no="2">
        <item>Disc CD-RW</item>
        <quantity>20</quantity>
        <price>2.95</price>
      </line>
    </lines>
  - <customer>
      <name>Technical University of Lublin</name>
      <street>Nadbystrzycka 38</street>
      <city>Lublin</city>
      <post_code>20-501</post_code>
    </customer>
  - <payment>
      <card_issuer>Master Card</card_issuer>
      <card_number>1234 567890 12345</card_number>
      <expiration_date month="10" year="2005" />
    </payment>
</customer_order>
```

https://www.researchgate.net/figure/The-orders-XML-file-example_fig1_266284839

Structure of XML data: elements

- The fundamental construct in an XML document is the **element**
- Element pair of matching start- and end-tags and all the text that appears between them
- XML documents must have a single root element
- Abbreviated notion: <element/>

XML example

```
<?xml version="1.0" encoding="UTF-8" ?>
- <customer_order number="004985" date="2004-06-24">
  - <lines>
    - <line no="1">
        <item>Disc CD</item>
        <quantity>30</quantity>
        <price>0.95</price>
      </line>
    - <line no="2">
        <item>Disc CD-RW</item>
        <quantity>20</quantity>
        <price>2.95</price>
      </line>
    </lines>
  - <customer>
      <name>Technical University of Lublin</name>
      <street>Nadbystrzycka 38</street>
      <city>Lublin</city>
      <post_code>20-501</post_code>
    </customer>
  - <payment>
      <card_issuer>Master Card</card_issuer>
      <card_number>1234 567890 12345</card_number>
      <expiration_date month="10" year="2005" />
    </payment>
</customer_order>
```

https://www.researchgate.net/figure/The-orders-XML-file-example_fig1_266284839

Structure of XML data: nesting

- Elements in an XML document must **nest properly**
- Text is said to appear **in the context of an element** if it appears between the start-tag and end-tag of that element
- Tags **are properly nested if** every start-tag has a unique matching end-tag that is in the context of the same parent element

- Nested representations are widely used in XML data interchange applications to avoid joins

XML example

```
<?xml version="1.0" encoding="UTF-8" ?>
- <customer_order number="004985" date="2004-06-24">
  - <lines>
    - <line no="1">
        <item>Disc CD</item>
        <quantity>30</quantity>
        <price>0.95</price>
      </line>
    - <line no="2">
        <item>Disc CD-RW</item>
        <quantity>20</quantity>
        <price>2.95</price>
      </line>
    </lines>
  - <customer>
      <name>Technical University of Lublin</name>
      <street>Nadbystrzycka 38</street>
      <city>Lublin</city>
      <post_code>20-501</post_code>
    </customer>
  - <payment>
      <card_issuer>Master Card</card_issuer>
      <card_number>1234 567890 12345</card_number>
      <expiration_date month="10" year="2005" />
    </payment>
</customer_order>
```

https://www.researchgate.net/figure/The-orders-XML-file-example_fig1_266284839

Structure of XML data: attributes

- Attributes are strings and do not contain markup
- Attributes can appear only once in a given tag
  - unlike subelements, which may be repeated.

- In general, it is advised to use attributes only to represent identifiers

XML example

```
<?xml version="1.0" encoding="UTF-8" ?>
- <customer_order number="004985" date="2004-06-24">
  - <lines>
    - <line no="1">
        <item>Disc CD</item>
        <quantity>30</quantity>
        <price>0.95</price>
      </line>
    - <line no="2">
        <item>Disc CD-RW</item>
        <quantity>20</quantity>
        <price>2.95</price>
      </line>
    </lines>
  - <customer>
      <name>Technical University of Lublin</name>
      <street>Nadbystrzycka 38</street>
      <city>Lublin</city>
      <post_code>20-501</post_code>
    </customer>
  - <payment>
      <card_issuer>Master Card</card_issuer>
      <card_number>1234 567890 12345</card_number>
      <expiration_date month="10" year="2005" />
    </payment>
</customer_order>
```

https://www.researchgate.net/figure/The-orders-XML-file-example_fig1_266284839

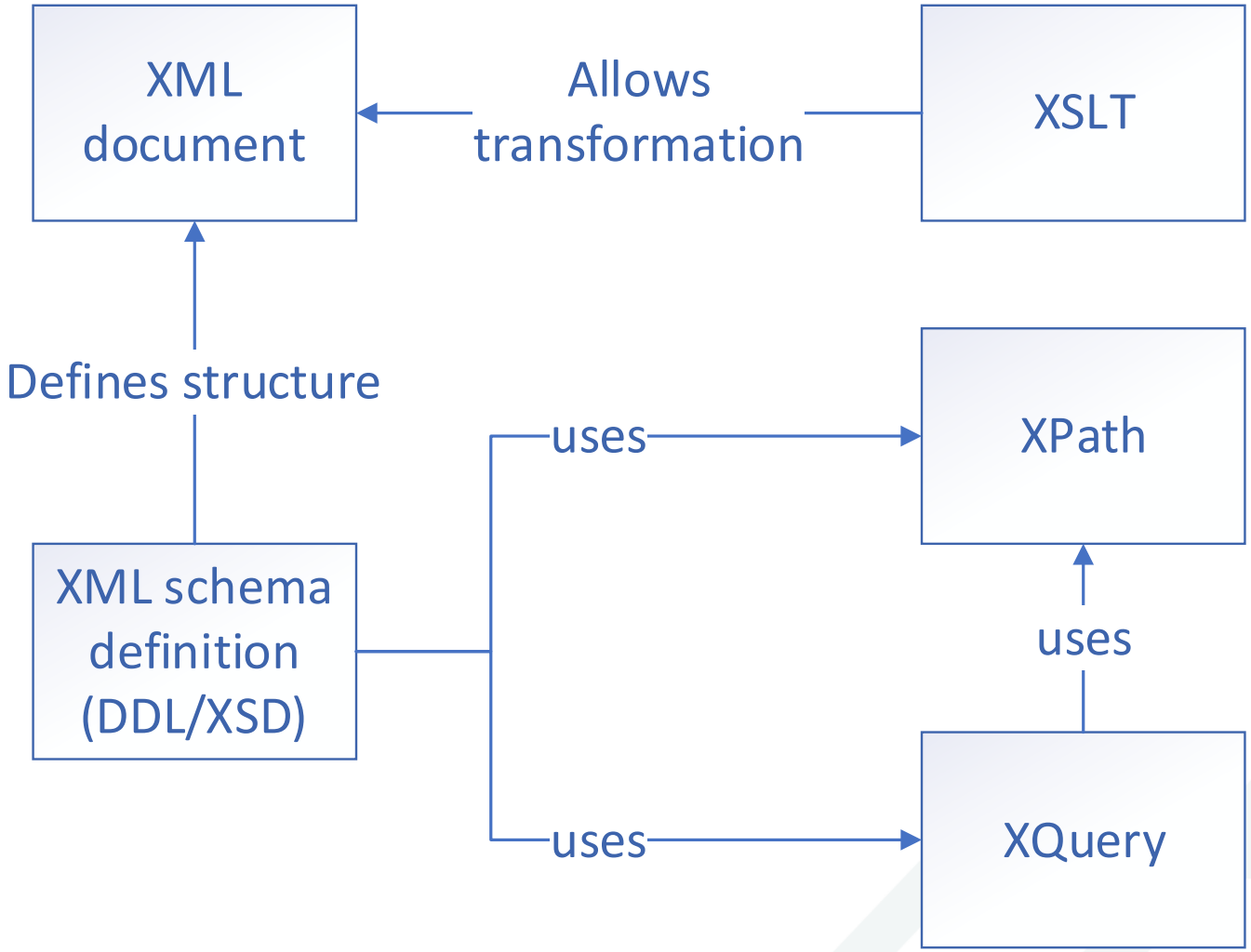
Structure of XML data: namespaces

- Allow organizations to specify globally unique names to be used as element tags in documents
- Used as universal resource identifier
- A **default namespace** can be defined by using the attribute xmlns in the root element

XML example

```
<?xml version="1.0" encoding="UTF-8" ?>
- <customer_order number="004985" date="2004-06-24">
  - <lines>
    - <line no="1">
        <item>Disc CD</item>
        <quantity>30</quantity>
        <price>0.95</price>
      </line>
    - <line no="2">
        <item>Disc CD-RW</item>
        <quantity>20</quantity>
        <price>2.95</price>
      </line>
  </lines>
- <customer>
    <name>Technical University of Lublin</name>
    <street>Nadbystrzycka 38</street>
    <city>Lublin</city>
    <post_code>20-501</post_code>
  </customer>
- <payment>
    <card_issuer>Master Card</card_issuer>
    <card_number>1234 567890 12345</card_number>
    <expiration_date month="10" year="2005" />
  </payment>
</customer_order>
```

Languages related to XML



Schema definition languages

How is schema defined in different DBMSs?

Document type definition

- The first schema-definition language included as part of the XML standard
- The main purpose of a DTD: to constrain and type the information present in the document
- Constrains only the appearance of subelements and attributes within an element

DTD example and corresponding XML

```
<!DOCTYPE university-3 [
    <!ELEMENT university ( (department|course|instructor)+)
    <!ELEMENT department ( building, budget )>
    <!ATTLIST department
        dept_name ID #REQUIRED >
    <!ELEMENT course (title, credits )>
    <!ATTLIST course
        course_id ID #REQUIRED
        dept_name IDREF #REQUIRED
        instructors IDREFS #IMPLIED >
    <!ELEMENT instructor ( name, salary )>
    <!ATTLIST instructor
        IID ID #REQUIRED
        dept_name IDREF #REQUIRED >
    ... declarations for title, credits, building,
        budget, name and salary ...
] >
```

```
<university-3>
    <department dept_name="Comp. Sci.">
        <building> Taylor </building>
        <budget> 100000 </budget>
    </department>
    <department dept_name="Biology">
        <building> Watson </building>
        <budget> 90000 </budget>
    </department>
    <course course_id="CS-101" dept_name="Comp. Sci"
                                instructors="10101 83821">
        <title> Intro. to Computer Science </title>
        <credits> 4 </credits>
    </course>
    <course course_id="BIO-301" dept_name="Biology"
                                instructors="76766">
        <title> Genetics </title>
        <credits> 4 </credits>
    </course>
    <instructor IID="10101" dept_name="Comp. Sci.">
        <name> Srinivasan </name>
        <salary> 65000 </salary>
...
```

XML Schema

- More sophisticated schema language
- Defines a number of built-in types such as string, integer, decimal date, and boolean
- Allows user-defined types:
  - simple types with added restrictions,
  - complex types constructed using constructors such as complexType and sequence

Namespace prefix

- To avoid conflicts with user-defined tags, prefix the XML Schema tag with the namespace prefix “xs:” is used
- This prefix is associated with the XML Schema namespace by the xmlns:xs
- Specification in the root element:

  <xs:schema xmlns:xs=“http://www.w3.org/2001/XMLSchema”>

- All types defined by XML Schema must be prefixed by this namespace prefix

Namespace example

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="university" type="universityType" />
<xs:element name="department">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="dept_name" type="xs:string"/>
            <xs:element name="building" type="xs:string"/>
            <xs:element name="budget" type="xs:decimal"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
<xs:element name="course">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="course_id" type="xs:string"/>
            ...
```

References, keys and key references

- References to previously defined types corresponding to **object references** can be made
- Keys and key references corresponding to the **primary-key and foreign-key** definition in SQL can be made

- For both: must specify a scope within which values are unique and form a key

- The **selector** is a path expression that defines the scope for the constraint
- **Field** declarations specify the elements or attributes that form the key.

Reference examples

- Primary key for department elements

```
<xs:key name = "deptKey">
      <xs:selector xpath = "/university/department"/>
      <xs:field xpath = "dept_name"/>
</xs:key>
```

- Foreign key constraint:

```
<xs: name = "courseDeptFKey" refer="deptKey">
      <xs:selector xpath = "/university/course"/>
      <xs:field xpath = "dept_name"/>
</xs:keyref>
```

Creating XML files

- XML data files: extension *.xml
- XML Schema files: extension *.xsd

XML document validation

- The process of checking a document written in XML to confirm
  - it is both well-formed
  - that it follows a defined structure

XML Editors

- Simpler options:
  – Notepad++

- More complex, rich tools that allow automatically create schema
  – Oxygen XML Editor

XML querying languages

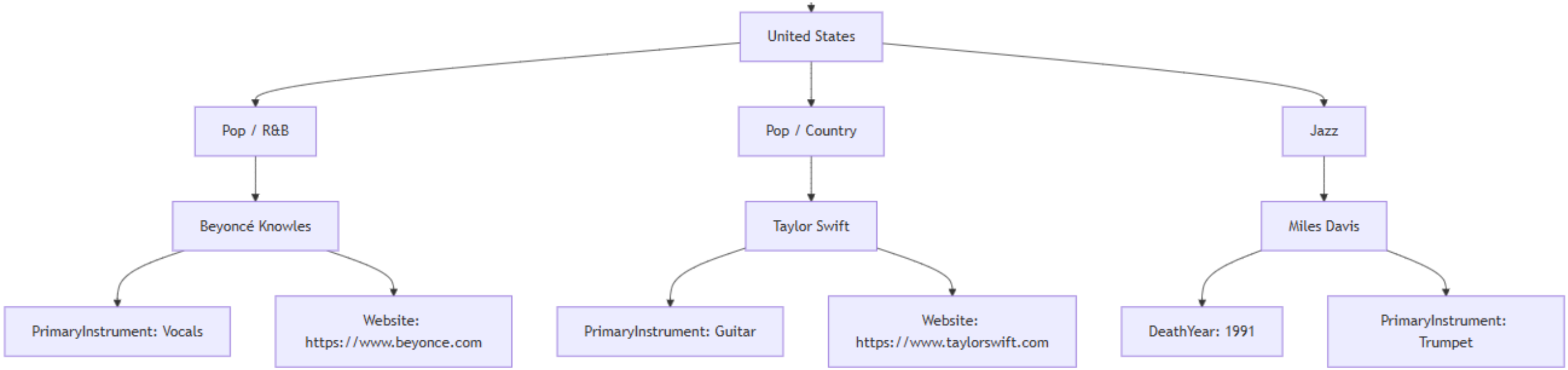
Querying languages

- XPath
- Xquery

- Both part of XML standard

XPath

- Every XPath query refers to a document
- Is a path expression



XPath Expression

- Describes a set of paths in a document
- Returns a sequence of nodes
- Evaluated in a **context**
- Absolute (starting at document root) or relative
- Consists of steps separated by /

Example XML - Book

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<catalog>
   <book id="bk101">
      <author>Gambardella, Matthew</author>
      <title>XML Developer&amp;apos;s Guide</title>
      <genre>Computer</genre>
      <price>44.95</price>
      <publish_date>2000-10-01</publish_date>
      <description>An in-depth look at creating applications
      with XML.</description>
   </book>
   <book id="bk102">
      <author>Ralls, Kim</author>
      <title>Midnight Rain</title>
      <genre>Fantasy</genre>
      <price>5.95</price>
      <publish_date>2000-12-16</publish_date>
   </book>
</catalog>
```

Example

doc(“book.xml”)/catalog/book/author

```
1 <author>Gambardella, Matthew</author>
2 <author>Ralls, Kim</author>
```

```
1  <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2  <catalog>
3     <book id="bk101">
4        <author>Gambardella, Matthew</author>
5        <title>XML Developer&apos;s Guide</title>
6        <genre>Computer</genre>
7        <price>44.95</price>
8        <publish_date>2000-10-01</publish_date>
9        <description>An in-depth look at creating applications
10       with XML.</description>
11    </book>
12    <book id="bk102">
13       <author>Ralls, Kim</author>
14       <title>Midnight Rain</title>
15       <genre>Fantasy</genre>
16       <price>5.95</price>
17       <publish_date>2000-12-16</publish_date>
18    </book>
19 </catalog>
```

XPath Expressions (Attributes)

doc(“book.xml”)/catalog/book/@id/string()

```
1 "bk101"

2 "bk102"
```

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<catalog>
   <book id="bk101">
      <author>Gambardella, Matthew</author>
      <title>XML Developer&apos;s Guide</title>
      <genre>Computer</genre>
      <price>44.95</price>
      <publish_date>2000-10-01</publish_date>
      <description>An in-depth look at creating applications
      with XML.</description>
   </book>
   <book id="bk102">
      <author>Ralls, Kim</author>
      <title>Midnight Rain</title>
      <genre>Fantasy</genre>
      <price>5.95</price>
      <publish_date>2000-12-16</publish_date>
   </book>
</catalog>
```

XPath Expressions (Axes)

- Other axes
  - Parent (“..”)
  - Ancestor
  - Descendant
  - Next-sibling (to the right)
  - Previous-sibling (to the left)
  - Self (“.”)
  - Descendant-or-self (“//”)

AxisName	Result
ancestor	Selects all ancestors (parent, grandparent, etc.) of the current node
ancestor-or-self	Selects all ancestors (parent, grandparent, etc.) of the current node and the current node itself
attribute	Selects all attributes of the current node
child	Selects all children of the current node
descendant	Selects all descendants (children, grandchildren, etc.) of the current node
descendant-or-self	Selects all descendants (children, grandchildren, etc.) of the current node and the current node itself
following	Selects everything in the document after the closing tag of the current node
following-sibling	Selects all siblings after the current node
namespace	Selects all namespace nodes of the current node
parent	Selects the parent of the current node
preceding	Selects all nodes that appear before the current node in the document, except ancestors, attribute nodes and namespace nodes
preceding-sibling	Selects all siblings before the current node
self	Selects the current node

XPath Expressions (Axes)

doc(“book.xml”)//author

```
1 <author>Gambardella, Matthew</author>
2 <author>Ralls, Kim</author>
```

```
1  <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2  <catalog>
3     <book id="bk101">
4        <author>Gambardella, Matthew</author>
5        <title>XML Developer&apos;s Guide</title>
6        <genre>Computer</genre>
7        <price>44.95</price>
8        <publish_date>2000-10-01</publish_date>
9        <description>An in-depth look at creating applications
10       with XML.</description>
11    </book>
12    <book id="bk102">
13       <author>Ralls, Kim</author>
14       <title>Midnight Rain</title>
15       <genre>Fantasy</genre>
16       <price>5.95</price>
17       <publish_date>2000-12-16</publish_date>
18    </book>
19 </catalog>
```

Other Expressions

- Wildcards: “*”
  - for any tag or attribute
  - E.g. doc(“/db/book.xml”)/catalog/*
- Conditions Boolean expressions within square brackets
  - TAG [i] : true only for the i-th child of the parent TAG
  - TAG [T] : true if TAG element has one or more sub-elements with tag T
  - TAG [@A] : true if TAG element has an attribute A

Thank you!